

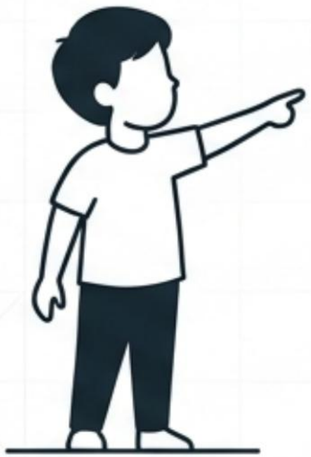
Fundamentos da Orientação a Objetos em Java

Do raciocínio abstrato à arquitetura de software (Prof. **Ronaldo Borges**)

NotebookLM

O Raciocínio Orientado a Objetos é Natural

Nós já pensamos em POO:
Quando crianças, aprendemos a observar o mundo e agrupar coisas com características semelhantes em conceitos simples.



O Processo de Abstração:
A mente humana extrai os detalhes essenciais para definir um modelo geral.



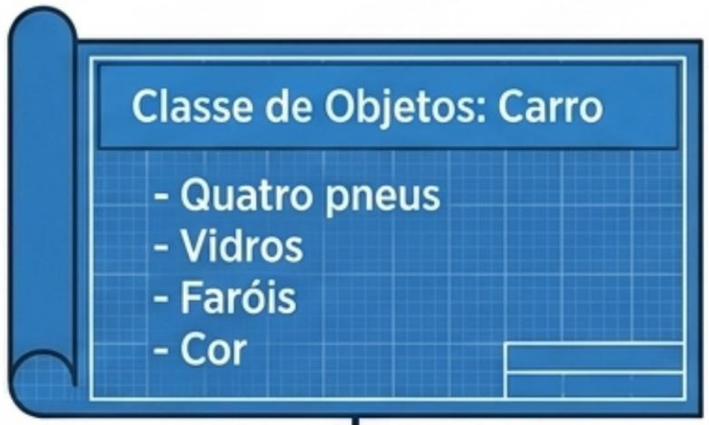
- Pessoa
- Carro
- Casa

Na programação, essa ação é exatamente o que chamamos de **Abstração**.

NotebookLM

O que é uma Classe?

A essência da linguagem Java. Uma classe é um modelo abstrato que define a forma, os dados e o código de um objeto.





A Regra: Cada carro na rua é um Objeto. Cada Objeto é uma Instância da Classe Carro.

NotebookLM

O Molde e a Memória

O Molde



A Classe: São planos lógicos que especificam como será o objeto.

> *Nota técnica: É uma abstração lógica pura. Não ocupa espaço de dados útil até ser instanciada.*

O Biscoito



O Objeto: A entidade física/digital gerada a partir do molde.

> *Nota técnica: Somente quando um objeto é criado (instanciado), ele passa a ter uma representação física na memória RAM.*

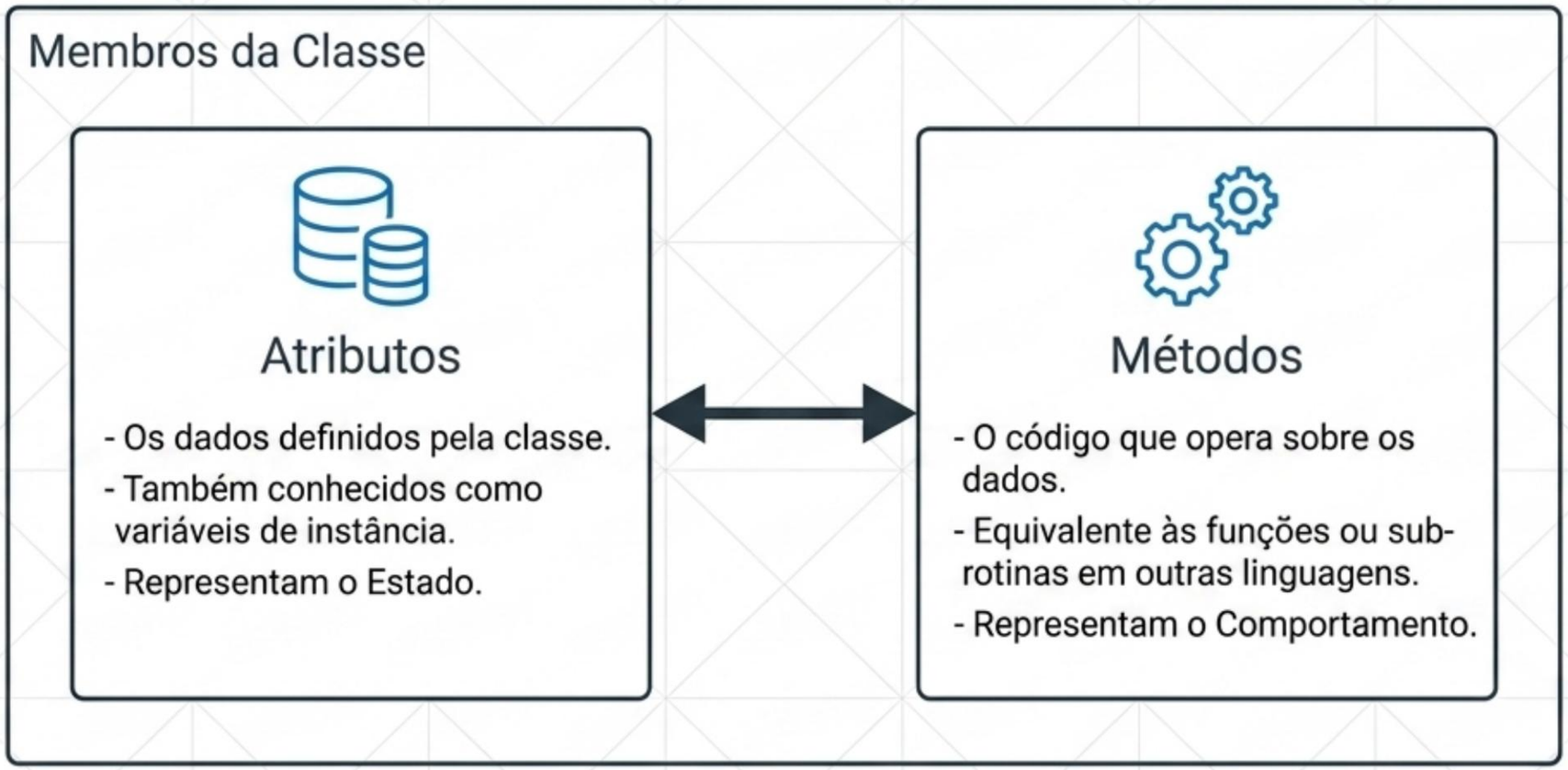
NotebookLM

Matriz de Identidade: Classe vs. Objeto

EU SOU CLASSE	EU SOU OBJETO
Sou um conceito abstrato.	Sou uma entidade concreta.
Defino o formato e o comportamento.	Armazeno estados (dados) reais.
Existo no código-fonte (.java).	Existo na memória durante a execução (runtime).
Objetos são minhas instâncias.	Sou a materialização da Classe.

NotebookLM

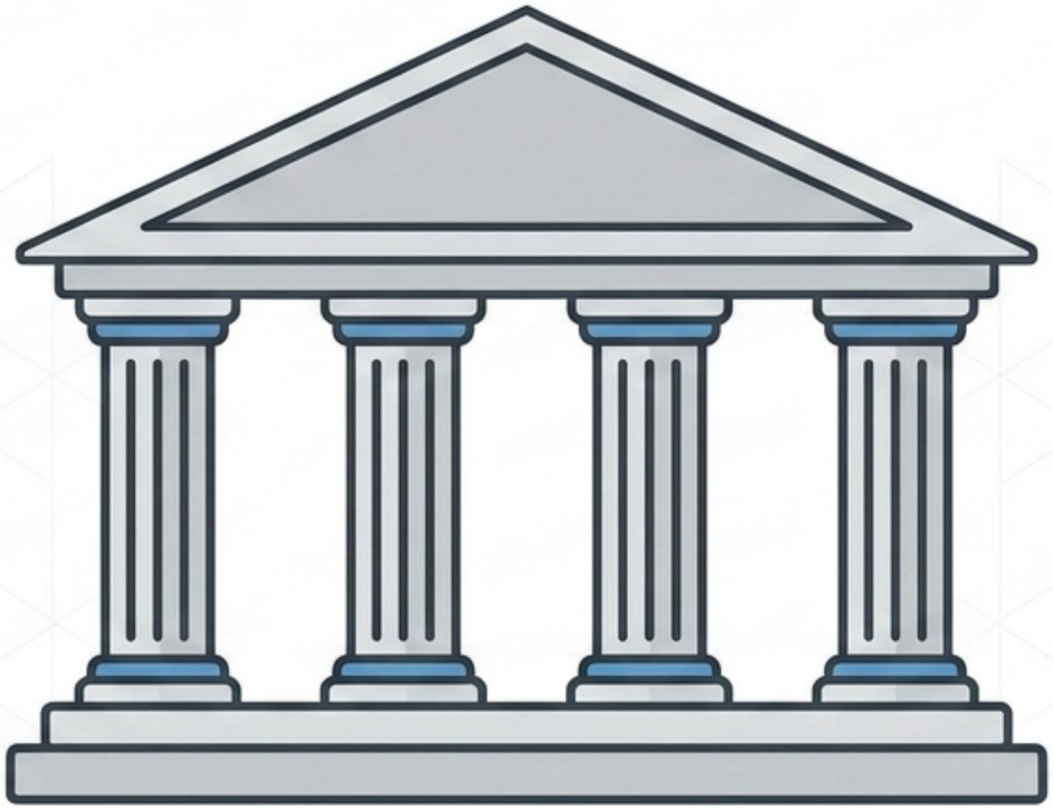
A Anatomia de uma Classe



A regra de ouro da POO: Dados controlando o acesso ao código.

NotebookLM

Os 4 Pilares da Programação Orientada a Objetos

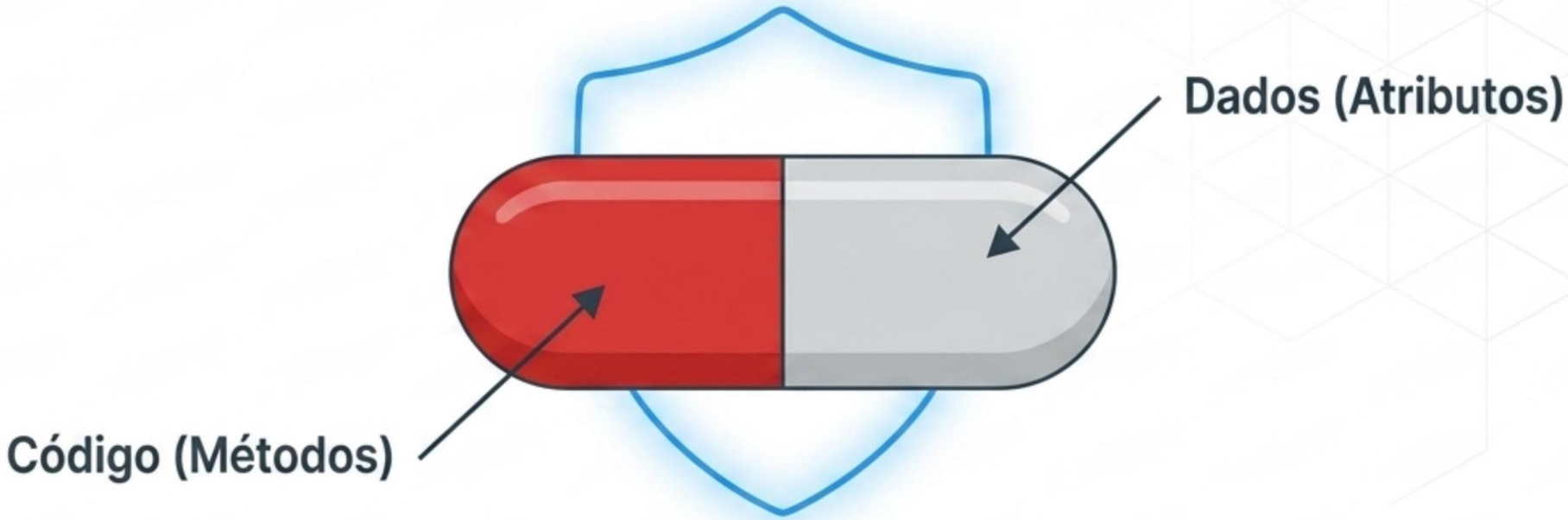


Encapsulamento Herança Polimorfismo Abstração

(Schildt, 2015) - A fundação para sistemas modulares, robustos e reutiliz

NotebookLM

Pilar 1: Encapsulamento



- A **Cápsula Protetora**: Mecanismo que vincula o código aos dados que ele trata.
- O **Objetivo**: Tornar os dados seguros contra interferência externa e uso indevido. O acesso direto aos dados (membros private) é bloqueado para os clientes da classe.
- A classe apresenta apenas seus serviços públicos (interface pública) para interação.

NotebookLM

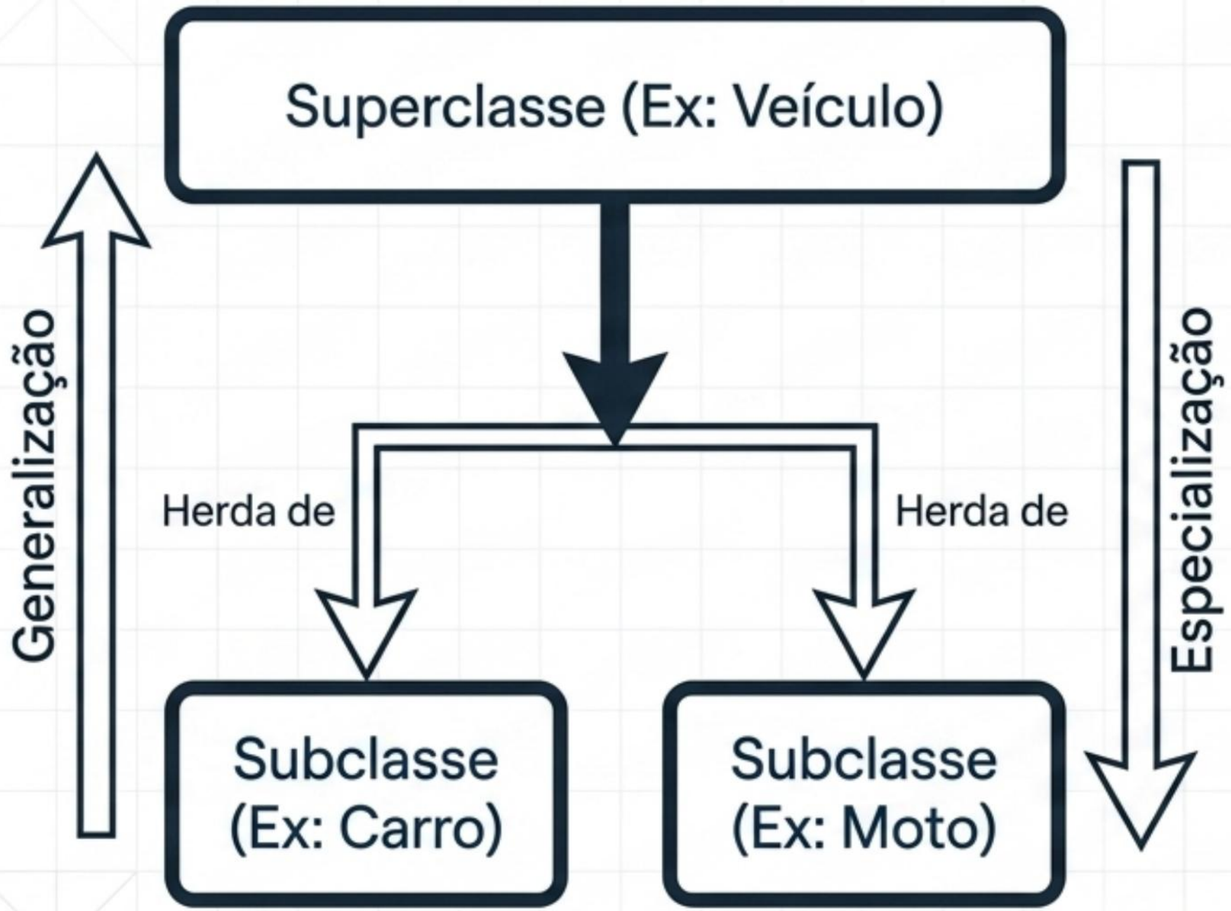
Matriz de Modificadores de Acesso (Visibilidade)

Modificador de Acesso	Mesma Classe	Mesmo Pacote	Subclasse	Mundo (Global)
private (Proteção máxima)	✔	✖	✖	✖
package / default	✔	✔	✖	✖
protected	✔	✔	✔	✖
public (Acesso irrestrito)	✔	✔	✔	✔

✔ = Acesso Permitido | ✖ = Acesso Bloqueado. O encapsulamento básico opera mantendo dados private e métodos public.

NotebookLM

Pilar 2: Herança



- **O Processo:** Uma nova classe é criada a partir de uma classe pré-existente.

- Um objeto adquire as propriedades e comportamentos de outro.

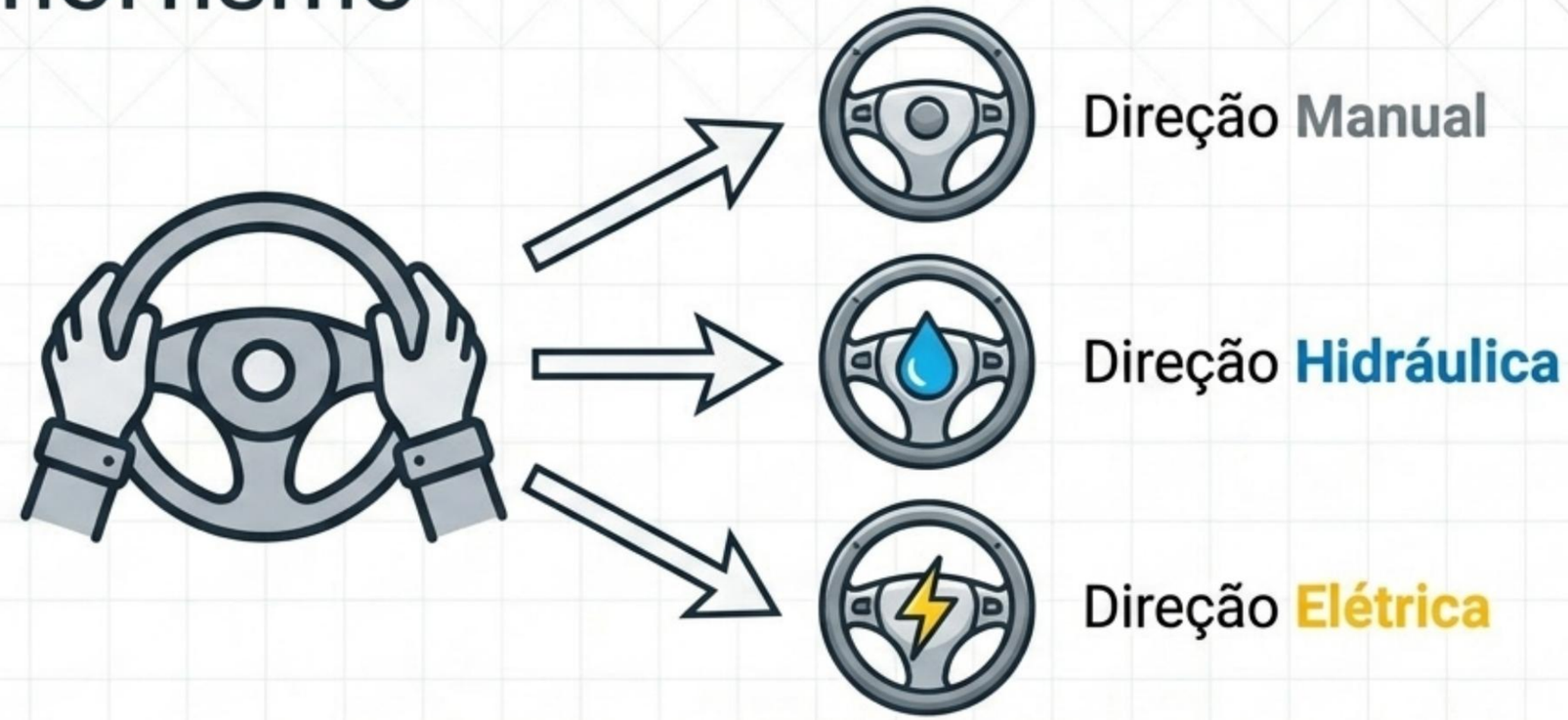
- A Dinâmica: Trabalha no eixo de Generalização (subir para a superclasse) e Especialização (descer para a subclasse).

- Vantagem: Elimina redundância de código através do compartilhamento de um design comum.

NotebookLM

Pilar 3: Polimorfismo

Ação do Motorista
(Método “Virar”)



- O Lema: **Uma interface, vários métodos.**
- Permite que métodos sejam chamados da mesma forma na superclasse, mas com execuções completamente diferentes na subclasse.
- Exemplo: O motorista aciona a Direção da mesma forma, mas o sistema decide se a resposta física será manual, hidráulica ou elétrica em tempo de execução.

NotebookLM

As Três Formas do Polimorfismo

Sobrescrita (Override)

Reescrevemos o método de uma superclasse na subclasse.

Ocorre entre classes diferentes conectadas por herança.

Substitui o comportamento padrão.

Sobrecarga (Overload)

Reescrevemos um método já existente na mesma classe.

O método mantém o mesmo nome, mas possui assinaturas (parâmetros) diferentes.

O compilador decide qual usar baseado nos argumentos.

Generalização (Downcasting)

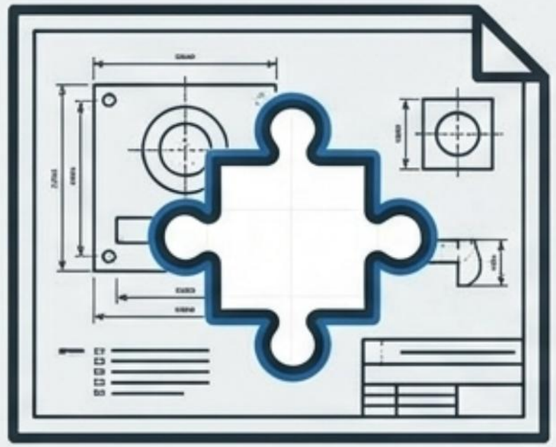
Coerção de um objeto para um tipo mais genérico (sua superclasse).

Permite que variáveis abstratas da superclasse manipulem objetos concretos de qualquer subclasse derivada.

Essencial para arrays polimórficos.

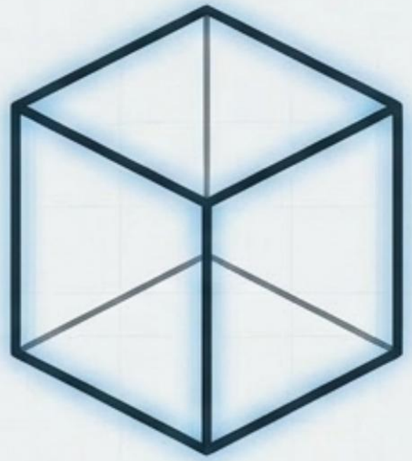
NotebookLM

Pilar 4: Abstração e Interfaces



Classes Abstratas

- Não podem ser instanciadas (são modelos incompletos).
- Contêm métodos abstratos (sem implementação) e métodos concretos.
- Serve como uma superclasse base para compartilhar um design comum.

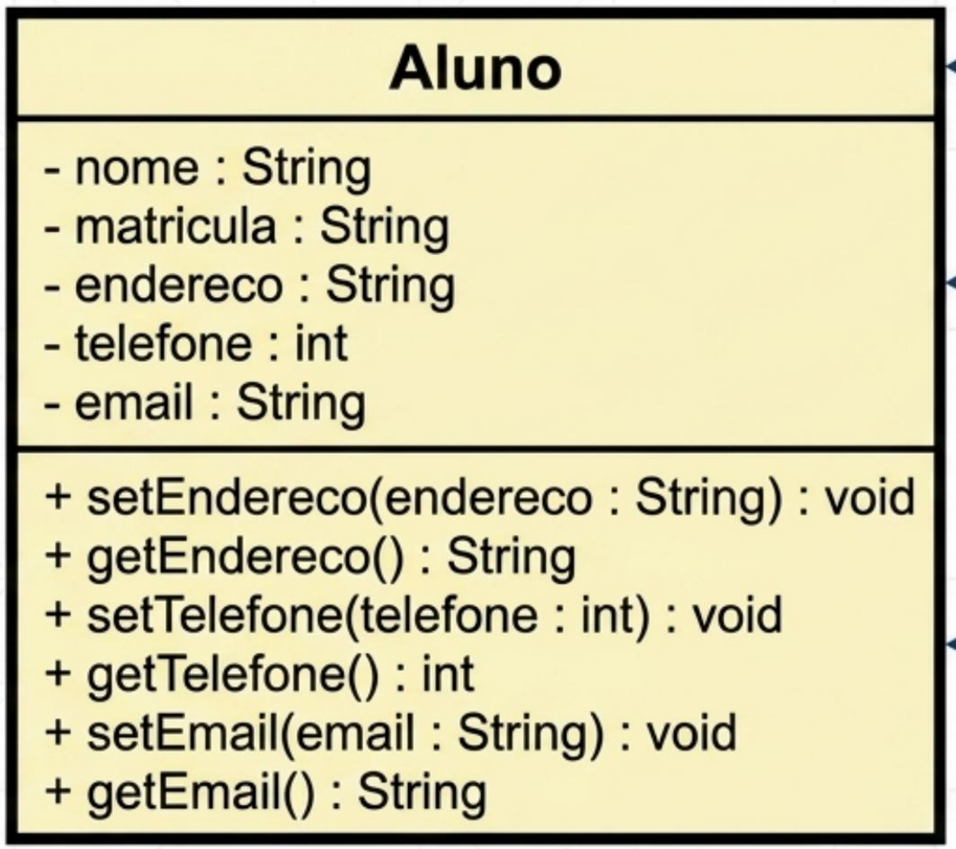


Interfaces

- Define o que a classe deve fazer, mas nunca como fará.
- Separação 100% da interface e da implementação.
- Uma classe realiza (implementa) uma interface (Ex: Payable).

NotebookLM

UML: O Diagrama de Classe Padrão



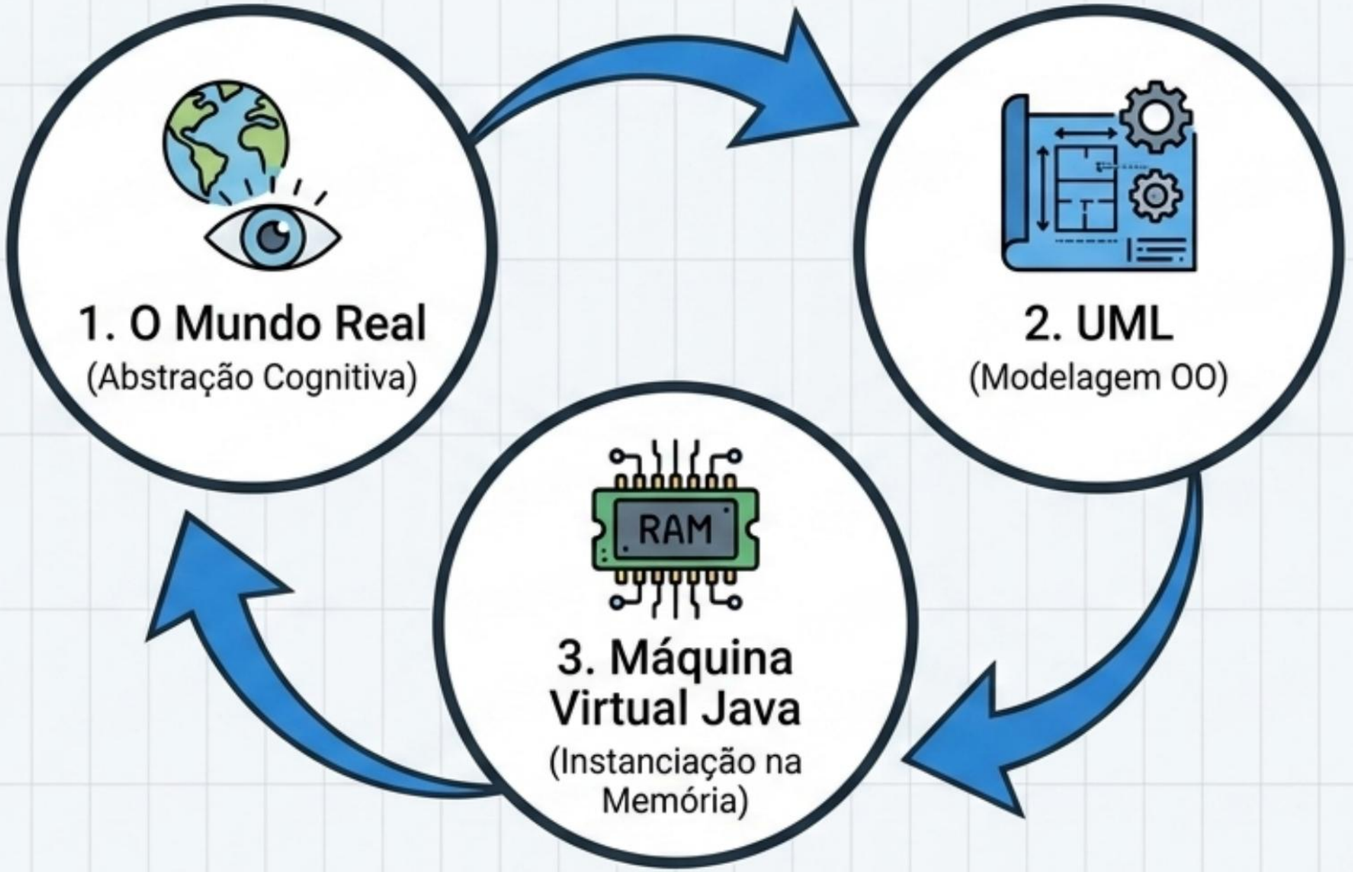
Compartimento Superior: O nome da classe (Aluno) centralizado e em negrito.

Compartimento Central (Atributos): Variáveis de instância. O símbolo (-) indica visibilidade private.

Compartimento Inferior (Operações): Os métodos. O símbolo (+) indica visibilidade public. Inclui parâmetros e tipo de retorno.

NotebookLM

O Ciclo da Programação Orientada a Objetos



O paradigma OO reduz a complexidade de sistemas permitindo que o código reflita entidades reais. Através do Encapsulamento, Herança, Polimorfismo e Abstração, criamos componentes de software consistentes, intuitivos e reutilizáveis. Ao dominar a Classe e a Instância, você domina a arquitetura de software moderna.

NotebookLM